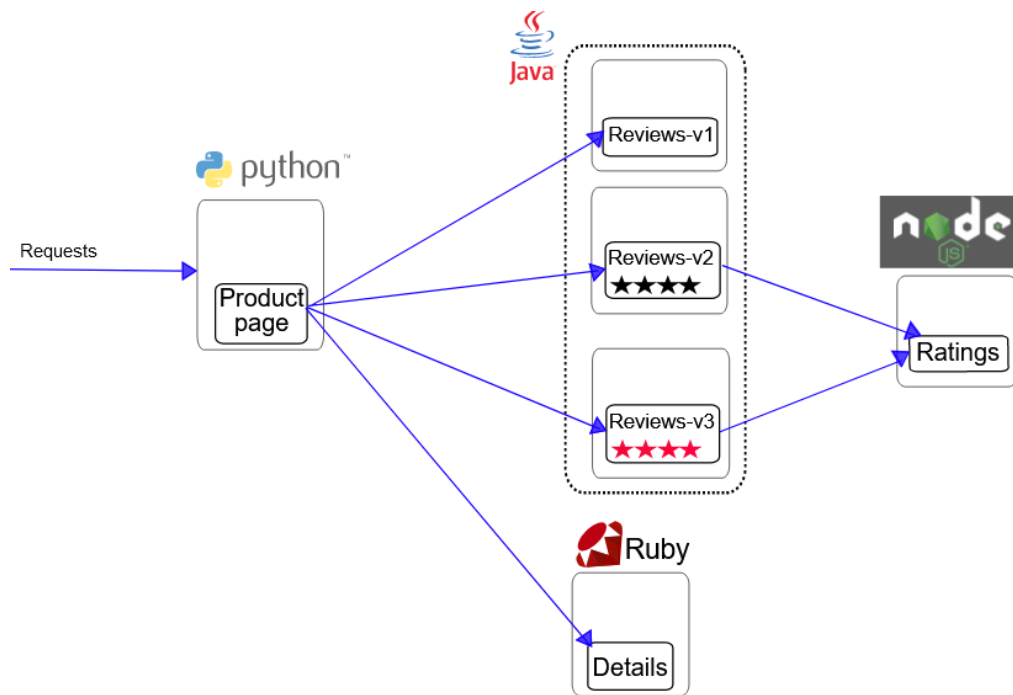




Istio Service Mesh Session – 1

Summary 8th April 2023

- **Microservices** is a software architecture style that structures an application as a collection of small, independent services, each of which performs a specific business function and communicates with other services through APIs (Application Programming Interfaces).
- **Example of microservices:**
 - The application displays information about a book. Displayed on the page is a description of the book, book details, and a few book reviews.
 - The Bookinfo application is broken into four separate microservices:
 - The product page microservice calls the details and reviews microservices to populate the page.
 - The details microservice contains book information.
 - The reviews microservice contains book reviews. It also calls the ratings microservice.
 - **There are 3 versions of the reviews microservice:**
 1. Version v1 doesn't call the ratings service.
 2. Version v2 calls the ratings service, and displays each rating as 1 to 5 black stars.
 3. Version v3 calls the ratings service, and displays each rating as 1 to 5 red stars.



➤ This application is **polyglot**, i.e., the microservices are written in different languages.

- A **deployment strategy** is a process or set of processes used to release a software application or service to its users. There are several deployment strategies that organizations can use, depending on their specific needs and requirements.
- **Canary deployment** is used in the above example for managing traffic.
- The purpose of a **canary deployment** is to reduce the risk of deploying a new version that impacts the workload. The method will incrementally deploy the new version, making it visible to new users in a slow fashion. As you gain confidence in the deployment, you will deploy it to replace the current version in its entirety.
- We need a web-based graphical user interface (GUI) for managing microservices. Using this GUI we can visualize and monitor the traffic within the microservices, making it easier for administrators to understand and manage the complex interactions between services. And that tool is known as **Kiali**.
- The purpose of Kiali is to provide a user-friendly interface for powerful traffic management, security, and observability features. With Kiali,

administrators can easily visualize the track traffic flow between services. They can also monitor the performance and health of each service and identify any issues or bottlenecks.

- The purpose of **Jaeger** is to provide **distributed tracing** for microservices-based applications. It allows you to track the flow of requests across multiple services and provides detailed insights into the performance of each service. Jaeger can help you identify and troubleshoot issues in your application quickly, allowing you to maintain high availability and reliability.
- Microservices have gained popularity due to their flexibility, scalability, and maintainability, but they also present unique challenges. Some of the challenges faced by the microservices world are:
 - 1) With microservices, it becomes difficult to keep track of all the services and their dependencies.
 - 2) Managing traffic between services can become difficult in a microservices-based application.
 - 3) With microservices, securing each service becomes important, as a single compromised service can bring down the entire application.
 - 4) With microservices, it becomes challenging to monitor and troubleshoot issues in real time.
- **Service Mesh** is the solution to all of the above challenges.
 - 1) Service discovery tools can help address to keep track of all the services challenges by providing a centralized registry of all the services and their endpoints.
 - 2) Service mesh can help address traffic between services challenges by providing traffic management features like canary deployment, load balancing, circuit breaking, and fault injection.
 - 3) Service mesh can help address securing each service challenge by providing security features like mutual TLS, RBAC, and secure service-to-service communication.
 - 4) Service mesh can help address monitor and troubleshoot challenges by providing features like distributed tracing, metrics, and logging.
- **Istio** is an open-source service mesh platform that provides a way to connect, secure, and manage microservices in a distributed system.

- **Architecture:**

- Istio uses a **sidecar multi-container architecture** to implement its service mesh. In this architecture, each microservice in the mesh is deployed as a container along with a **sidecar proxy container**, such as **Envoy**. The sidecar proxy acts as a communication gateway for the microservice, handling all network traffic to and from the microservice.
- Istio provides a centralized management system for the service mesh and control all the proxy program and that is known as **control tower**.

